

PHASE 2 — SCOPE OF WORK

Master BMS Development

Advanced Features & Extended Firmware Modules

Document Title	Phase 2 — Scope of Work:Master BMS Advanced Features
Project	BMS Controller Development (Post Rev One)
Prepared By	Dreamfly Innovations Pvt. Ltd.
Version	v1.0 — Initial Scope Definition
Date	13 April 2026
Status	Proposed / Awaiting Approval

1. Overview

This document defines the Scope of Work for Phase 2 of the BMS Controller Development programme. Phase 2 builds directly upon the Rev One hardware and core firmware delivered in Phase 1, adding advanced communication protocols, power management capabilities, and wireless connectivity features. The four primary modules in this scope are:

- OTA (Over-The-Air) Firmware Update via CAN Bus with mid-transfer fault protection
- DBC File generation for CAN 2.0 frame decoding (Charger, CAN Analyzer, Busmaster)
- Automatic CAN Bus Switchover between CAN 2.0 and DroneCAN (UAVCAN v1)
- nRF52832 BLE External Module Integration via adapter PCB on the EXTERNAL UART port
- BMS Deep-Sleep Mode with intelligent wake-up logic and watchdog supervision

Additional enhancements incorporated: - Enhanced State of Health (SoH) calculation and reporting including capacity fade, internal resistance, and cycle count - Cell-level Overvoltage and Undervoltage indications - Charging threshold current alert.

2. Module 1 — OTA Firmware Update over CAN Bus

2.1 Feature Description

The BMS shall support secure, resumable Over-The-Air (OTA) firmware updates delivered over the CAN 2.0B or DroneCAN transport layer. The bootloader and application firmware shall together implement a robust multi-block transfer protocol that survives power interruptions, CAN bus dropouts, or host-side failures at any point during the update session.

2.2 Functional Requirements

Requirement ID	Description
OTA-FR-01	Bootloader shall receive firmware binary partitioned into fixed-size CAN frames (≤ 8 bytes payload per frame, or up to 64 bytes with CAN FD).
OTA-FR-02	Transfer session shall be identified by a unique Session ID; duplicate frames shall be detected and discarded.
OTA-FR-03	Each received block shall be written to a dedicated QSPI flash staging partition before any overwrite of the active application.
OTA-FR-04	A running CRC-32 checksum shall be validated block-by-block; the bootloader shall request retransmission of any block failing CRC.
OTA-FR-05	On completion of the full binary transfer, a final CRC-32 over the entire image shall be verified before committing the image as active.
OTA-FR-06	If power is lost or CAN bus drops mid-transfer, the staging partition shall retain the partial image; on next power-up the bootloader shall resume from the last successfully written block (resume byte offset communicated via handshake frame).
OTA-FR-07	The active application shall never be erased until the staging image passes full CRC validation. A fallback rollback to the previous firmware shall be triggered on three consecutive failed boot attempts.
OTA-FR-08	Transfer status (progress %, block index, error code) shall be broadcast on a dedicated CAN diagnostic frame at 1 Hz during the update session.
OTA-FR-09	A host-initiated ABORT command frame shall cleanly terminate the session and restore the BMS to normal operational mode without firmware corruption.

2.3 Protocol State Machine

The OTA transfer follows a well-defined state machine across both the host tool and the BMS bootloader:

- IDLE → Awaiting OTA_START command from host
- SESSION_OPEN → Session ID acknowledged; host begins sending DATA_BLOCK frames
- BLOCK_RX → Per-block CRC check; ACK or NACK with expected block index returned to host
- INTERRUPTED → Power/CAN loss detected on reconnect; bootloader broadcasts RESUME_REQ with last good offset
- VERIFY → All blocks received; full-image CRC-32 validation in progress
- COMMIT → Image written to active flash partition; reboot initiated
- ROLLBACK → Failed boot counter exceeded threshold; previous image restored from backup slot

2.4 Deliverables

- ◆ Bootloader firmware module: OTA session handler, block receiver, QSPI staging write logic
- ◆ Host-side PC script (Python): OTA sender with progress bar, retry logic, and session resume
- ◆ CAN frame spec document: all OTA frame IDs, DLC, payload maps
- ◆ Test report: power-cycle mid-transfer recovery, partial-image protection, rollback validation

3. Module 2 — DBC File for CAN 2.0 Frame Decoding

3.1 Feature Description

A CAN Database (DBC) file shall be authored covering all CAN 2.0B frames transmitted and received by the BMS, including frames for the charger interface, real-time telemetry, fault reporting, and configuration. The DBC shall be validated in Vector CANalyzer, Busmaster (open-source), and compatible tools.

3.2 Frame Coverage

Frame Category	CAN ID (Example)	Signals Covered
BMS Status & Telemetry	0x18FF50E5	Pack voltage, pack current, SoC (%), SoH (%), temperature (max/avg/min)
Cell Voltages (Multi-frame)	0x18FF51E5	Individual cell voltages (all cells), cell balancing flags
Fault & Protection Status	0x18FF52E5	OVP, UVP, OCP, OTP, UTP, short circuit, communication fault bitmask
Charger Command TX	0x1806E5F4	Charge voltage setpoint, charge current limit, charger enable flag
Charger Status RX	0x18FF50E5	Output voltage, output current, charger fault code, hardware protection flag
DroneCAN Bridge Frame	0x18FF53E5	Forwarded UAVCAN BatteryInfo fields mapped to CAN 2.0B for analyzer visibility
OTA Control Frames	0x18FF60E5	Session ID, block index, CRC, OTA status code (per Module 1)
Configuration R/W	0x18FF70E5	Parameter ID, value (read/write), ACK/NACK response

3.3 DBC File Specification Requirements

- All signals shall define: name, start bit, length, byte order (Intel/Motorola), factor, offset, min, max, unit, and receiving node.
- Enumerated signals (fault bitmask, charger status codes, OTA state) shall use Value Definitions (VAL_) for human-readable decoding.

- Message attributes shall include send type (cyclic/event), cycle time (ms), and transmitting node name (e.g., BMS_NODE, CHARGER_NODE).
- DBC shall be validated in Busmaster (open-source) with a trace capture confirming all frames decode without warnings.
- A companion signal decode guide (PDF) shall be provided listing every signal with description, resolution, and engineering range.

3.4 Deliverables

- ◆ BMS_CAN_Database.dbc — validated DBC file
- ◆ Signal_Decode_Reference.pdf — engineering reference for all signals
- ◆ Busmaster workspace (.bws) pre-configured with BMS DBC loaded
- ◆ Validation trace screenshots from Busmaster and CANalyzer confirming correct decoding

4. Module 3 — Automatic CAN Bus Switchover (CAN 2.0 ↔ DroneCAN)

4.1 Feature Description

The BMS firmware shall implement an intelligent, automatic CAN bus protocol switchover mechanism that detects the active CAN network type at startup and during runtime, selecting between CAN 2.0B (standard extended frame, 29-bit ID) and DroneCAN (UAVCAN v1) operation without requiring manual reconfiguration or firmware reflash.

4.2 Detection & Switching Logic

Stage	Description
Startup Detection (Primary)	On power-up, the BMS listens for 500 ms on the CAN bus. Presence of DroneCAN NodeStatus frames (data type ID 341) identifies a DroneCAN network. Absence defaults to CAN 2.0B mode.
Runtime Detection (Secondary)	If no valid frames are received in the active protocol for >3 seconds, the firmware switches protocol mode and re-probes for 500 ms before committing to the new mode.
Manual Override via CLI	Operator may force a specific protocol via the UART/USB-C CLI using 'can mode can20' or 'can mode dronecan' commands. The selection is stored in QSPI config flash and survives reboot.
Status Indicator	The active CAN mode shall be reflected in BMS status frames and accessible via the CLI 'status' command for diagnostic visibility.
Switchover Latency	Protocol switchover including re-initialisation of the CAN peripheral and protocol stack shall complete within 1 second with no packet loss on the newly active protocol.

4.3 DroneCAN Node Behaviour During CAN 2.0 Mode

When CAN 2.0B mode is active, the DroneCAN node stack shall be suspended (not de-initialised) to enable rapid re-activation if a DroneCAN network is subsequently detected. The CAN 2.0B telemetry frames shall follow the frame map defined in the DBC file (Module 2). MODBUS RTU operation (RS-485 interface) is unaffected by CAN mode selection.

4.4 Deliverables

- Firmware module: `can_switchover.c/.h` with protocol detection FSM and re-init routines
- CLI command extension: 'can mode', 'can status', 'can probe' commands with help strings
- Configuration flash schema update: CAN mode preference field
- Test report: auto-detection on DroneCAN-only network, CAN 2.0-only network, and mixed bus with switchover event

5. Module 4 — nRF52832 BLE External Module Integration

5.1 Feature Description

A dedicated Bluetooth Low Energy (BLE) communication channel shall be added to the BMS using the Nordic Semiconductor nRF52832-QFAA-G-R7 SoC, mounted on a compact adapter PCB that plugs into the EXTERNAL UART header of the BMS controller board. The module shall broadcast a defined BMS telemetry payload at 1 Hz, enabling real-time monitoring via any BLE-capable device (smartphone, laptop, tablet) without requiring physical wiring.

5.2 Adapter PCB Specification

Parameter	Specification
SoC	Nordic Semiconductor nRF52832-QFAA-G-R7 (ARM Cortex-M4F, 512 KB Flash, 64 KB RAM, integrated BLE 5.0)
Host Interface	UART (3.3V logic, 115200 baud, 8-N-1) via 4-pin connector matching EXTERNAL UART pinout on BMS board: VCC, GND, TX, RX
Antenna	PCB trace antenna or U.FL connector for external whip antenna (layout TBD based on enclosure constraints)
PCB Form Factor	Compact module: ≤30 × 20 mm, single-sided SMD assembly for ease of rework
Power Supply	3.3V supplied from BMS EXTERNAL UART header VCC rail; total current draw < 20 mA in active advertising mode
Regulatory	Module to use CE/FCC pre-certified nRF52832 SoC; antenna design shall avoid affecting certification

5.3 BLE Advertising Payload (1 Hz Broadcast)

The nRF52832 shall operate in non-connectable undirected advertising mode at 1 Hz, broadcasting a custom manufacturer-specific data payload within the BLE advertisement packet. The payload shall be structured as follows:

Byte Offset	Length (B)	Field Name	Description
0–1	2	Pack Voltage	Pack voltage in mV (uint16, LSB first)
2–3	2	Pack Current	Pack current in mA signed (int16, LSB first), positive = discharge
4	1	SoC	State of Charge in % (0–100, uint8)
5	1	SoH	State of Health in % (0–100, uint8)
6	1	Max Cell Temp	Maximum cell temperature in °C offset +40 (uint8, range –40 to +215 °C)
7	1	Min Cell Temp	Minimum cell temperature in °C offset +40 (uint8)
8–9	2	Fault Bitmask	Active fault flags (uint16): OVP[0], UVP[1], OCP[2], OTP[3], UTP[4], SCP[5], COM[6], others reserved
10	1	BMS State	Operational state enum: 0=Idle, 1=Charging, 2=Discharging, 3=Balancing, 4=Fault, 5=Sleep
11	1	Sequence Counter	Rolling 8-bit counter incremented each broadcast; used by receiver to detect missed packets

5.4 UART Communication (BMS ↔ nRF52832)

The BMS host firmware shall transmit a compact binary frame to the nRF52832 at 1 Hz over UART. The nRF52832 firmware (Nordic SDK / Zephyr RTOS) parses this frame and places the payload into the BLE advertisement data buffer before the next advertising interval. The nRF52832 requires no BLE connection establishment — it is a pure beacon.

- ◆ UART frame: SOF (0xAA 0x55) | Length (1B) | Payload (12B as per table above) | CRC-8 | EOF (0x55 0xAA)
- ◆ BMS firmware TX: driven from the existing 1 Hz system tick task; non-blocking DMA UART transmit
- ◆ nRF52832 RX: interrupt-driven UART RX with frame validation; invalid frames silently discarded

5.5 Deliverables

- Adapter PCB: KiCad schematic + layout, Gerbers, BOM (nRF52832-QFAA-G-R7, decoupling, connector)
- nRF52832 firmware: Zephyr / nRF5 SDK project — UART receiver, BLE advertiser, payload packer
- BMS host firmware module: ble_uart_tx.c/h — 1 Hz payload serialiser and DMA UART transmit
- BLE Scanner test tool: Python script (bleak library) to receive and decode advertisements for validation
- Integration test report: confirmed broadcast at 1 Hz, payload accuracy vs CAN telemetry, range test (open air)

6. Module 5 — BMS Deep-Sleep Mode

6.1 Feature Description

The BMS shall implement a deep-sleep power mode that activates automatically when the system enters an Idle state (no active charge or discharge current above threshold, no active faults, no active communication session). During deep-sleep, quiescent current shall be minimised while a hardware watchdog and dedicated wake-up sources maintain system supervisory integrity. The BMS shall wake fully within a defined latency upon detection of any configured wake event.

6.2 Sleep Entry Conditions

The firmware state machine shall transition to Deep-Sleep when ALL of the following conditions are simultaneously true:

- ▶ BMS operational state = IDLE (no charging, no discharging detected for a configurable timeout period, default: 120 seconds)
- ▶ All cell voltages within safe resting range (no overvoltage or undervoltage flags active)
- ▶ Pack temperature within safe resting range (no overtemperature or undertemperature flags active)
- ▶ No active CAN, MODBUS, or UART communication session in progress
- ▶ External WAKE button not held depressed (prevents entry if operator is actively interacting)

6.3 Deep-Sleep Architecture

Sub-System	Behaviour During Deep-Sleep
STM32 Core	Enters STOP 2 mode (lowest-power stop mode with RTC and IWDG active). All non-essential clocks gated. GPIO outputs tri-stated or held at safe levels.
Independent Watchdog (IWDG)	Continues running from LSI oscillator (~32 kHz). IWDG timeout set to 2× the expected wake-check interval. Firmware shall pet the IWDG during the brief periodic wake window to confirm system is alive.
Periodic Wake Window	The RTC alarm wakes the MCU every configurable interval (default: 10 seconds) for a brief supervisory window (< 100 ms): read current sensor, check cell voltages, pet IWDG, re-enter sleep if no wake event.
ACS37612 Current Sensor	Remains powered at reduced sample rate via ADC in low-power mode. Current threshold comparator interrupt configured to assert on $ I >$ configurable threshold (e.g., 1 A) to trigger immediate wake.
CAN Transceiver	Placed in Silent/Sleep mode (TXD held recessive, RXD monitored). CAN activity detection via GPIO interrupt on RXD line to trigger wake if bus traffic detected.
NTC Temperature Sensing	Powered down during sleep; sampled during periodic wake window only.
BLE Module (nRF52832)	Transitions to nRF52832 System OFF mode; no BLE advertising during BMS deep-sleep. Wakes with BMS on WAKE event.

Sub-System	Behaviour During Deep-Sleep
QSPI Flash Logging	Final log entry written before sleep entry (SleepEntry event, timestamp, SoC, voltage). No writes during sleep.
Status LED (if present)	Single short blink every 10 seconds during deep-sleep to indicate system is alive (driven from RTC periodic wake window).

6.4 Wake-Up Sources

Wake Source	Latency Target	Description
Charge/Discharge Current Detect	< 200 ms	ACS37612 output crosses $\pm$ threshold $\rightarrow$ ADC comparator interrupt $\rightarrow$ EXTI line asserts $\rightarrow$ MCU wakes from STOP 2
External WAKE Button	< 200 ms	Dedicated GPIO EXTI line configured as FALLING EDGE interrupt; hardware debounce via RC filter (10 ms) + firmware debounce
CAN Bus Activity	< 500 ms	CAN transceiver RXD line monitored via EXTI; any frame on bus triggers wake and re-enables CAN stack within 500 ms
RTC Periodic Alarm	N/A — scheduled	Supervisory periodic wake every 10 s (default, configurable via CLI). Not a user-visible event; used for IWDG service and background health check only.
Voltage Threshold Breach	< 500 ms	Detected during RTC periodic wake window; if cell voltage moves out of safe resting range the system fully wakes and re-evaluates state machine

6.5 Sleep Exit Sequence

On wake-up from any source, the firmware shall execute the following ordered sequence:

1. Re-enable all peripheral clocks and restore GPIO configurations
2. Pet the IWDG immediately to prevent inadvertent reset during start-up
3. Log WakeEvent record to QSPI flash (timestamp, wake source, SoC, voltage)
4. Re-enable ADC, current sensor, and CAN transceiver
5. Run a rapid BMS sanity check (cell voltages, temperatures, fault flags)
6. Assert WAKE to nRF52832 via UART to resume BLE advertising
7. Transition state machine to appropriate active state (IDLE, CHARGING, or DISCHARGING) based on current sensor reading

6.6 Configuration Parameters (CLI / MODBUS Accessible)

Parameter	Default Value	Description
sleep_idle_timeout_s	120 s	Idle duration before sleep entry is initiated
sleep_current_threshold_mA	1000 mA	Current magnitude threshold for current-detect wake (absolute value)
sleep_periodic_interval_s	10 s	RTC periodic wake interval for supervisory checks and IWDG service
sleep_enable	1	Master enable/disable for deep-sleep feature (0 = always awake, for bench testing)

6.7 Deliverables

- Firmware module: power_manager.c/.h — sleep entry/exit FSM, wake source arbitration, IWDG service
- RTC driver extension: periodic alarm configuration and ISR
- EXTI configuration: WAKE button, CAN RXD monitor, ACS37612 comparator output
- CLI command: 'sleep enable/disable/status/config' with live parameter display
- MODBUS register map update: sleep parameters readable/writable via MODBUS RTU
- Power consumption measurement report: deep-sleep quiescent current (target: < 500 μ A total system)
- Test report: all five wake sources validated, IWDG pet confirmed, QSPI log entries verified

7. Phase 2 Deliverables Summary

#	Module	Deliverable Type	Description
1	OTA via CAN	Firmware + Script	Bootloader OTA module, host Python sender, CAN frame spec doc, test report
2	DBC File	Database + Doc	BMS_CAN_Database.dbc, Signal Decode Reference, Busmaster workspace, validation trace
3	CAN Switchover	Firmware + Report	can_switchover.c/.h, CLI extension, config flash update, test report
4	BLE Module (nRF52832)	Hardware + Firmware + Tool	Adapter PCB (KiCad + Gerbers + BOM), nRF52832 firmware, BMS UART TX module, Python scanner, test report
5	Deep-Sleep Mode	Firmware + Report	power_manager.c/.h, RTC/EXTI drivers, CLI/MODBUS extension, power measurement report,

#	Module	Deliverable Type	Description
			test report

8. Assumptions & Exclusions

8.1 Assumptions

- ✓ Rev One hardware (Phase 1) is fully functional and provides the CAN, UART, QSPI flash, and GPIO resources required by all Phase 2 modules.
- ✓ The EXTERNAL UART header on the BMS board is populated with a suitable 4-pin connector (VCC, GND, TX, RX) and provides 3.3V at ≥ 10 mA.
- ✓ QSPI flash capacity is sufficient for the OTA staging partition in addition to Phase 1 logging requirements (minimum 8 MB recommended).
- ✓ Phase 1 bring-up testing has been completed and baseline firmware is functional before Phase 2 development commences.
- ✓ The customer will provide access to a CAN bus network (DroneCAN node or CAN 2.0 analyser) for switchover testing.

9. Commercials

Phase 2 cost and timeline will be scoped and finalised following successful completion of Phase 1 Rev One bring-up, per the original phased proposal. A detailed effort estimate (man-days) for each module above will be presented in a separate commercial quotation.

A preliminary high-level estimate is provided below for planning purposes only and is subject to revision:

Module	Estimated Effort	Estimated Duration
OTA Firmware Update via CAN	10–15 man-days	2–3 weeks
DBC File Generation	3–5 man-days	1 week
CAN Auto-Switchover	5–8 man-days	1–2 weeks
nRF52832 BLE Integration	12–18 man-days	2–3 weeks
BMS Deep-Sleep Mode	8–12 man-days	2 weeks
Total (Preliminary)	38–58 man-days	8–11 weeks